

CHAPTER 04

OpenCV 주요기능

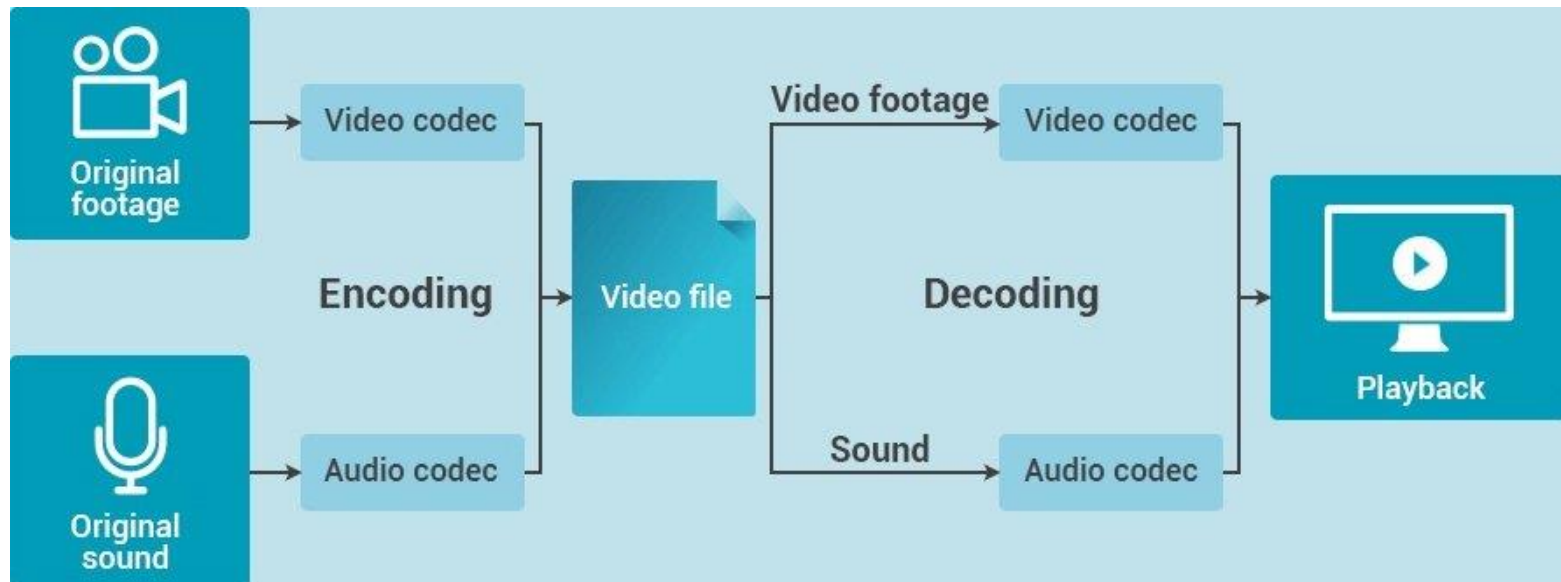
OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

4장 OpenCV 주요 기능

4.1 카메라와 동영상 파일 다루기

코덱(codec)

- 대부분의 동영상 파일은 고유의 코덱(codec)을 이용하여 압축된 형태로 저장됨(codec = encoder + decoder)
- 코덱은 복잡한 알고리즘을 이용하여 대용량 동영상 데이터를 압축하고 반대로 압축을 해제하여 프레임을 읽는 기능을 제공함
- OpenCV는 현재 널리 사용되고 있는 MPEG-4, H.264 등 코덱을 제공하기 때문에 VideoCapture 클래스를 사용하면 동영상 파일을 쉽게 불러와서 사용할 수 있음



VideoCapture 클래스: 객체 생성(동영상)

```
VideoCapture::VideoCapture(const String& filename, int apiPreference = CAP_ANY);  
bool VideoCapture::open(const String& filename, int apiPreference = CAP_ANY);
```

- `filename` 동영상 파일 이름
- `apiPreference` 사용할 비디오 캡처 API 백엔드
- 반환값 (VideoCapture::open() 함수) 열기가 성공하면 true, 실패하면 false

- OpenCV에서 동영상 파일을 다루는 방법은 앞서 살펴본 카메라 입력 처리하는 방법과 매우 비슷함
- VideoCapture 객체를 생성할 때 동영상 파일 이름을 지정함
- 디폴트 생성자로 VideoCapture 객체를 생성할 때는 VideoCapture::open() 멤버 함수를 호출해야 함

```
VideoCapture cap("video.mp4");
```

```
VideoCapture cap;
```

```
cap.open("video.mp4");
```

VideoCapture 클래스: 객체 생성(동영상)

- filename : *.avi, *.mpg, *.mp4 등 확장자를 갖는 동영상 파일 이름을 전달함
 - 현재 프로젝트 폴더에 동영상 파일이 있으면 "video.mp4" 형태로 파일 이름만 지정,
 - 다른 폴더에 동영상 파일이 있다면 절대 경로 또는 상대 경로를 추가하여 파일 이름을 지정함 "c:/video/video.mp4" 또는 "c:\\video\\video.mp4"
 - C/C++언어에서 \는 탈출문자임 문자 \'로 사용하려면 \\로 표현해야 함
- apiPreference 인자에는 동영상 파일을 불러올 때 사용하는 API를 지정할 수 있음. VideoCaptureAPIs 열거형 상수 중 하나를 지정함
 - 대부분의 경우 apiPreference 인자를 생략하면 기본값인 CAP_ANY를 지정하며, 이 경우 시스템이 알아서 적절한 방법을 선택하여 사용함

VideoCapture 클래스: 객체 생성(동영상)

표 4-1 주요 VideoCaptureAPIs 열거형 상수

VideoCaptureAPIs 열거형 상수	설명
CAP_ANY	자동 선택
CAP_V4L, CAP_V4L2	V4L/V4L2(리눅스)
CAP_FIREWIRE, CAP_FIREWARE, CAP_IEEE1394	IEEE 1394 드라이버
CAP_DSHOW	다이렉트쇼(DirectShow)
CAP_PVAPI	PvAPI, Prosilica GigE SDK
CAP_OPENNI	OpenNI
CAP_MSMF	마이크로소프트 미디어 파운데이션 (Microsoft Media Foundation)
CAP_GSTREAMER	GStreamer
CAP_FFMPEG	FFMPEG 라이브러리
CAP_IMAGES	OpenCV에서 지원하는 일련의 영상 파일 (예) img_%02d.jpg
CAP_OPENCV_MJPEG	OpenCV에 내장된 MotionJPEG 코덱

VideoCapture 클래스: 객체 생성(동영상)

```
bool VideoCapture::isOpened() const;
```

• 반환값 카메라 또는 동영상 파일이 사용 가능하면 true, 그렇지 않으면 false

- VideoCapture::isOpened() 함수를 이용하여 객체 생성시 에러 체크

```
if (!cap.isOpened( )) {  
    cerr << "Video open failed!" << endl;  
    return -1;  
}
```

동영상 파일 처리하기 : 객체 생성

- 방법1: 동영상 파일을 다룰 때에도 우선 VideoCapture 객체를 생성하고 VideoCapture::open() 멤버 함수를 이용하여 파일 열기

```
VideoCapture cap;  
cap.open("video.mp4");
```

- 방법2: 동영상 파일 이름을 인자로 갖는 VideoCapture 생성자를 이용하여 객체 생성 및 파일 열기

```
VideoCapture cap("video.mp4");
```

- 동영상파일이 정상적으로 열렸는지는 VideoCapture::isOpened() 멤버 함수를 이용하여 확인해야 함

```
if (!cap.isOpened( )) {  
    cerr << "Video open failed!" << endl;  
    return -1;  
}
```


동영상 파일 처리하기: 재생 속도 계산

- 동영상 파일을 재생할때 동영상의 FPS(frames per second)값을 고려하지 않으면 동영상이 너무 빠르거나 느리게 재생됨
- 동영상을 적절한 속도로 재생하기위해 동영상의 FPS 값을 이용하여 매 프레임 사이의 지연 시간(delay)을 계산해야 함
- 지연 시간 값은 반복문안에서 waitKey() 함수의 인자로 사용됨
- 예: 초당 30프레임 동영상을 재생하는 delay값은 33ms이며, 이는 각 프레임을 33ms시간 간격으로 출력해야 함을 의미함

```
double fps = cap.get(CAP_PROP_FPS);  
int delay = cvRound(1000 / fps);
```

코드4-3 : 동영상 파일 처리

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    VideoCapture cap("stopwatch.avi");
    if (!cap.isOpened( )) {
        cerr << "Video open failed!" << endl;
        return -1;
    }
    cout << "Frame width: " << cvRound(cap.get(CAP_PROP_FRAME_WIDTH))
        << endl;
    cout << "Frame height:" << cvRound(cap.get(CAP_PROP_FRAME_HEIGHT))
        << endl;
    cout << "Frame count: " << cvRound(cap.get(CAP_PROP_FRAME_COUNT))
        << endl;
    // 전체 프레임수 출력
```

코드4-3 : 동영상 파일 처리

```
double fps = cap.get(CAP_PROP_FPS);
cout << "FPS: " << fps << endl;
int delay = cvRound(1000 / fps);
Mat frame, inversed;
while (true) {
    cap >> frame;
    if (frame.empty( )){ cerr << "frame error!" << endl; break; }

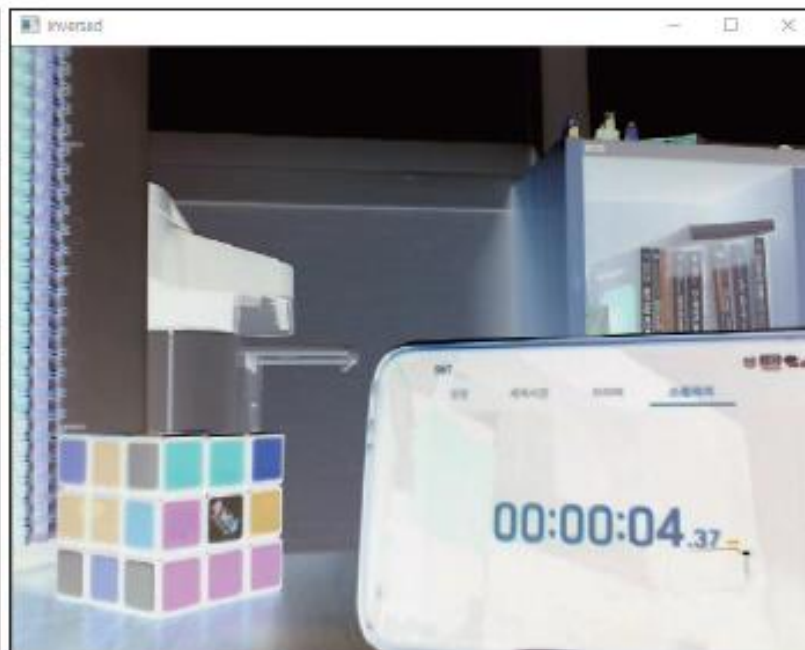
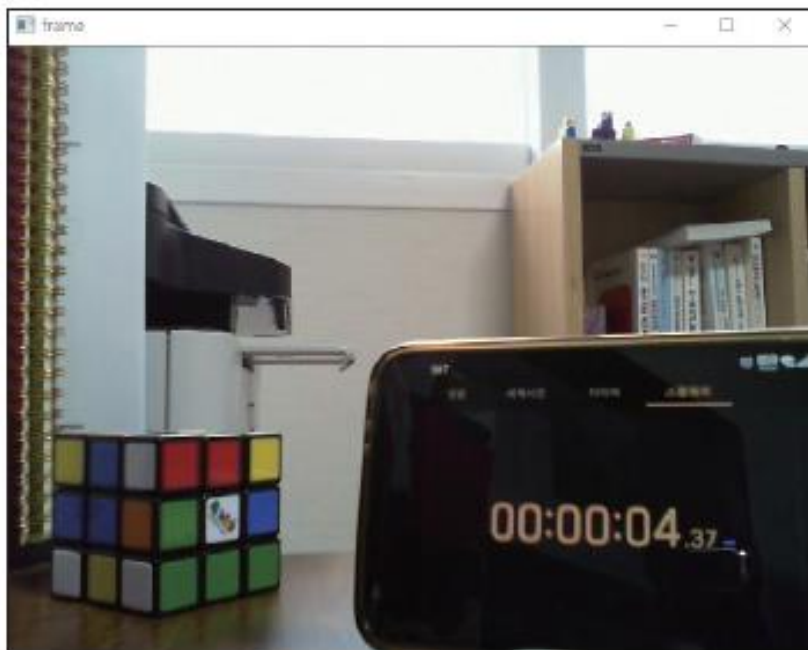
    inversed = ~frame;           // 영상반전
    imshow("frame", frame);
    imshow("inversed", inversed);

    if (waitKey(delay) == 27) break; // ESC key 누르면 종료
}
return 0;
}
```

코드4-3 : 동영상 파일 처리

```
C:\coding\opencv\ch04\video\#x64\Release\video.exe  
Frame width: 640  
Frame height: 480  
Frame count: 378  
FPS: 30
```

파일 전체 프레임 수



동영상 파일 저장하기

- OpenCV는 카메라 및 동영상 파일의 프레임을 출력하는 기능뿐만 아니라 연속된 프레임을 동영상 파일로 저장하는 기능도 제공함
- OpenCV에서 동영상 파일을 생성하고 프레임을 저장하기 위해서는 VideoWriter 클래스를 사용함

코드 4-4 간략화한 VideoWriter 클래스 정의

```
01  class VideoWriter
02  {
03  public:
04      VideoWriter();
05      VideoWriter(const String& filename, int fourcc, double fps,
06                  Size frameSize, bool isColor = true);
07      virtual ~VideoWriter();
08  }
```

동영상 파일 저장하기

```
VideoWriter::VideoWriter(const String& filename, int fourcc, double fps,  
                          Size frameSize, bool isColor = true);  
bool VideoWriter::open(const String& filename, int fourcc, double fps,  
                      Size frameSize, bool isColor = true);
```

- **filename** 저장할 동영상 파일 이름
- **fourcc** 동영상 압축 코덱을 표현하는 4-문자 코드
- **fps** 저장할 동영상의 초당 프레임 수
- **frameSize** 동영상 프레임의 가로 및 세로 크기
- **isColor** 이 값이 true이면 컬러 동영상으로 저장하고, false이면 그레이스케일 동영상으로 저장합니다. 이 플래그는 Windows 운영 체제에서만 지원합니다.
- **반환값** (VideoWriter::open() 함수) 열기가 성공하면 true, 실패하면 false

- 새로운 동영상 파일을 만들려면 먼저 VideoWriter 클래스 객체를 생성해야 함
- 기본생성자로 VideoWriter 객체를 생성한 경우 VideoWriter::open() 멤버 함수를 이용하여 저장할 동영상 파일을 쓰기 모드로 열어야 함

동영상 파일 저장하기

```
static int VideoWriter::fourcc(char c1, char c2, char c3, char c4);
```

- `c1, c2, c3, c4` 코덱을 표현하는 1byte 문자 네 개
- 반환값 정수형 4-문자 코드

- VideoWriter 생성자와 `open()` 함수의 두 번째 인자 `fourcc`는 4-문자 코드(four character code)의 약자이며 말 그대로 네 개의 문자로 구성된 코드임
- `fourcc`는 동영상 파일의 코덱, 압축 방식, 색상 혹은 픽셀 포맷 등을 정의하는 정수 값이며, 코덱을 표현하는 네 개의 문자를 묶어서 `fourcc`를 생성함
- `fourcc`에 해당하는 정수 값은 `VideoWriter::fourcc()` 함수를 사용하여 생성할 수 있음

동영상 파일 저장하기

- VideoWriter::fourcc() 함수를 사용하여 생성할 수 있는 주요 fourcc 코드와 관련 코덱 정보를 표 4-3에 정리함

fourcc 코드 생성 방법	코덱 설명
VideoWriter::fourcc('D', 'I', 'V', 'X')	DivX MPEG-4 코덱
VideoWriter::fourcc('X', 'V', 'I', 'D')	XVID MPEG-4 코덱
VideoWriter::fourcc('F', 'M', 'P', '4')	FFMPEG MPEG4 코덱
VideoWriter::fourcc('W', 'M', 'V', '2')	Windows Media Video 8 코덱
VideoWriter::fourcc('M', 'J', 'P', 'G')	모션 JPEG 코덱
VideoWriter::fourcc('Y', 'V', '1', '2')	YUV 4:2:0 Planar(비압축)
VideoWriter::fourcc('X', '2', '6', '4')	H.264/AVC 코덱
VideoWriter::fourcc('A', 'V', 'C', '1')	Advanced Video 코덱

동영상 파일 저장하기

- 예를 들어 DivX MPEG-4 코덱을 이용하는 output.avi 동영상 파일을 생성하려면 다음과 같은 형태로 코드를 작성함

```
VideoWriter video;  
int fourcc = VideoWriter::fourcc('D', 'I', 'V', 'X');  
video.open("output.avi", fourcc, fps, Size(w, h));
```

- 이 코드에서 fps는 FPS 값, w와 h는 프레임 크기를 나타냄
- VideoWriter 클래스는 VideoWriter::open() 함수와 같은 인자 구성을 갖는 생성자를 지원하기 때문에 앞에서 다음 처럼 간단히 작성 가능

```
VideoWriter video("output.mp4", fourcc, fps, Size(w, h));
```

동영상 파일 저장하기

```
VideoWriter& VideoWriter::operator << (const Mat& image);  
void VideoWriter::write(InputArray image);
```

• `image` 추가할 프레임

- 열려 있는 동영상 파일에 새로운 프레임을 추가하기 위해서는 << 연산자 함수 또는 VideoWriter::write() 함수를 사용함
- 프레임을 추가할 때, 새로 추가하는 image 프레임 크기는 동영상 파일을 생성할 때 지정했던 프레임 크기와 같아야 함
- 컬러로 설정된 동영상 파일에 그레이스케일 영상을 추가하면 정상적으로 저장되지 않으므로 주의해야 함

코드4-5: 동영상 파일로 저장

- 컴퓨터에 연결된 카메라로부터 프레임을 입력 받고, 해당 프레임을 반전한 영상을 이용하여 동영상 파일을 생성함
- VideoCapture 클래스를 이용하여 카메라 장치를 열고, VideoWriter 클래스를 이용하여 반전된 동영상 파일을 생성함

코드4-5: 동영상 파일로 저장

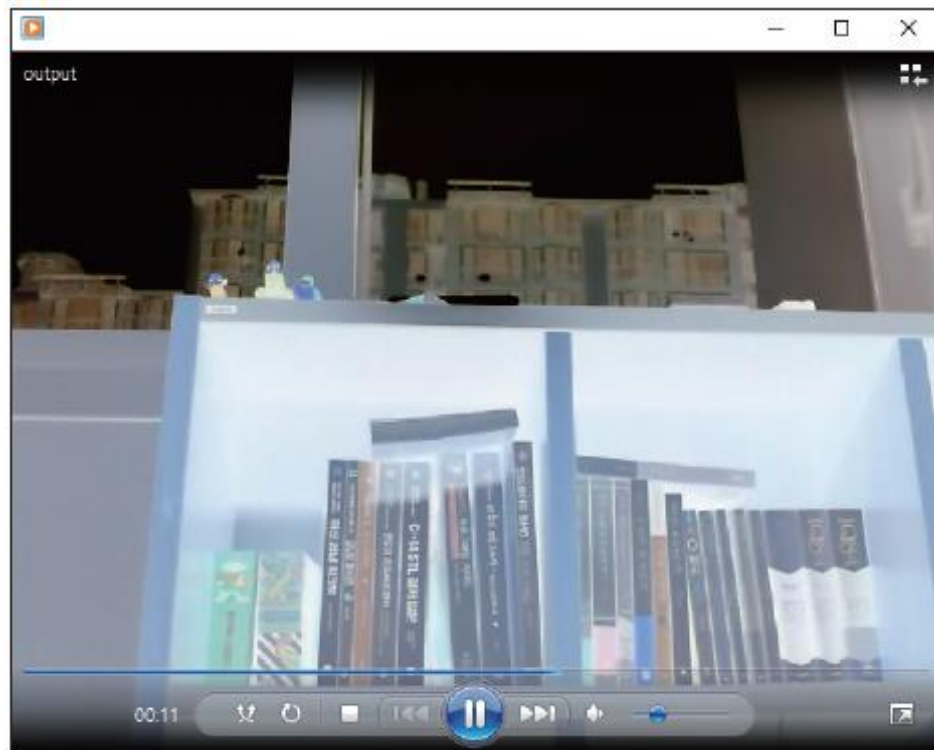
```
#include <iostream>
using namespace std;
#include <opencv2/opencv.hpp>
using namespace cv;
int main(void)
{
    VideoCapture cap(0);
    if (!cap.isOpened()) {
        cerr << "Camera open failed!" << endl;
        return -1;
    }
    int w = cvRound(cap.get(CAP_PROP_FRAME_WIDTH));
    int h = cvRound(cap.get(CAP_PROP_FRAME_HEIGHT));
    int fourcc = VideoWriter::fourcc('D', 'I', 'V', 'X');
```

코드4-5: 동영상 파일로 저장

```
VideoWriter outputVideo("output.avi", fourcc, 30, Size(w, h));  
                                // 프로젝트폴더에 동영상파일생성  
  
if (!outputVideo.isOpened()) {  
    cout << "File open failed!" << endl;  
    return -1;  
}  
  
Mat frame, inversed;  
while (true) {  
    cap >> frame;  
    if (frame.empty()) break;  
  
    inversed = ~frame;  
    outputVideo << inversed;                                // 반전 영상을 동영상 파일에 저장  
    imshow("frame", frame);  
    imshow("inversed", inversed);  
    if (waitKey(33) == 27) break;  
}  
return 0;  
}
```

코드4-5: 동영상 파일로 저장

- 다음은 저장된 output.avi 파일을 재생하는 화면임, 동영상 파일은 프로젝트 폴더에 생성된다.

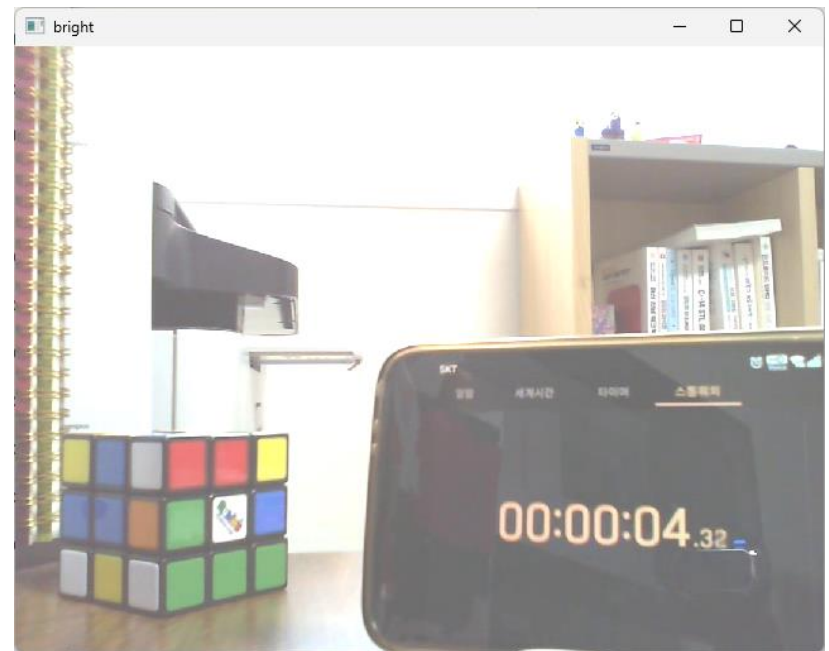
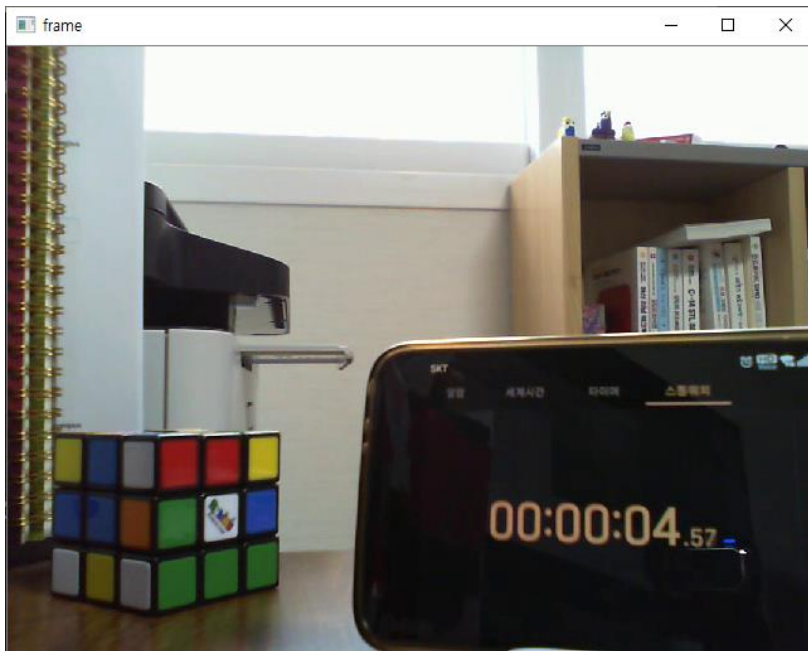


실습과제 1

- 다음 용어에 대하여 조사하시오.
 - MPEG4
 - Divx
 - Xvid
 - H264
 - H265

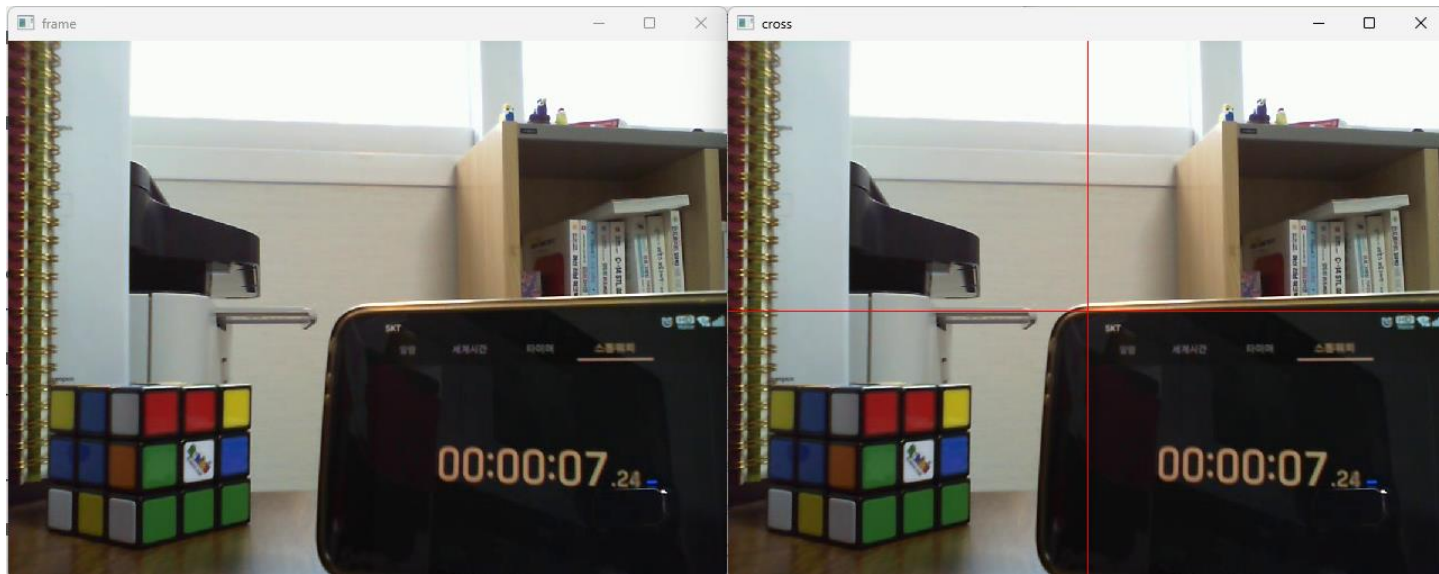
실습과제2

- 코드4-3의 동영상(stopwatch.avi)을 R,G,B값 모두 100만큼 증가시킨 후 원본과 결과영상을 모두 출력하라.
- 원본 영상과 처리 후 영상을 분리하여 저장할 것
- 무한루프로 작성하고 q or Q를 누르면 종료하도록 할 것



실습과제3

- 코드4-3의 동영상(stopwatch.avi)을 아래 그림처럼 라인을 그린 후 원본과 결과영상을 모두 출력하라.
- 출력영상은 동영상파일로(output.avi) 저장하라 -> 동영상을 재생하여 결과 확인할 것
- 원본 영상과 처리 후 영상을 분리하여 저장할 것
- 무한루프로 작성하고 q or Q를 누르면 종료하도록 할 것



output.avi	✓	2025-04-08 오후 3:29	AVI 파일	1,928KB
stopwatch.avi	✓	2019-02-24 오후 8:19	AVI 파일	3,596KB

실습과제4

- 카메라로 촬영한 영상을 동영상 파일로 저장하는 프로그램을 작성하라.
- Q 또는 q키를 누르면 프로그램을 종료하라.
- 프로젝트 폴더에 생성된 파일을 재생하여 결과를 확인할 것.

실습과제5

- 카메라로 촬영한 영상을 사용자가 원하는 시간 동안만 동영상을 저장하는 프로그램을 작성하라.
- 원하는 시점에서 s키를 누르면 동영상 파일로 저장을 시작하고 e키를 누르면 저장을 종료하고 프로그램을 종료하는 코드를 작성하라.
- 프로젝트 폴더에 생성된 파일을 재생하여 결과를 확인할 것.

과제 제출 방법

- 소스파일, 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출 하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.